

Manual del alumno

Todo el temario del curso, unidad por unidad: los conceptos de Python explicados con ejemplos de cartera, el código que necesitas, los problemas típicos con su solución y cómo usar la IA con criterio.

10 días

de curso

12 unidades

del programa

Datos

100 % sintéticos

Proyecto

final opcional

Arnau Ivern Sala

Data & AI Engineer · Profesor del curso

Este manual acompaña a los notebooks del repositorio del curso; no los sustituye. Los ejemplos usan los datasets sintéticos de `data/raw/` (ninguno contiene datos reales de pólizas, siniestros ni clientes). Versiones de referencia: Python 3.12+, pandas 2.3, NumPy 2.5, SciPy 1.18, scikit-learn 1.9.

Índice

0 Antes de empezar

0.1 Cómo está organizado el curso

0.2 Tu entorno de trabajo

0.3 Los datasets del curso

I El temario, unidad por unidad

U1 Entorno de trabajo e IA aplicada · Día 1

U2 Fundamentos de Python · Día 2

U3 Control de flujo · Día 3

U4 Funciones · Día 4

U5 Programación orientada a objetos · Día 5

U6 Código profesional · Día 6

U7 pandas y NumPy · Días 7 y 8

U8 Visualización · Día 8

U9 Bases de datos y APIs · Día 9

U10 Estadística aplicada · Día 9

U11 Machine learning aplicado · Día 10

U12 Cálculo actuarial-financiero · Día 10

PF El proyecto final

II Referencia

A Cómo usar la IA en el curso (y en tu trabajo)

B Guía de errores: síntoma → causa → solución

C Chuletas: Python, Excel → pandas, Git

D Glosario de Python

E Respuestas de la autoevaluación

F Cómo seguir aprendiendo

0 Antes de empezar

Este curso está pensado para ti: dominas la estadística, la probabilidad y la modelización actuarial, y quieres aprender Python para trabajar mejor, no para cambiar de profesión. Por eso los ejemplos hablan de pólizas, primas, siniestros y provisiones, y nunca se explica qué es una distribución de Poisson: se explica cómo se calcula en Python. Casi todo lo que te va a costar es **sintaxis y mecánica** del lenguaje. Es normal, y se supera practicando.

Cómo está organizado el curso

DÍA	UNIDAD(ES)	TEMA
1	U1	Entorno de trabajo, notebooks e introducción a Claude Code
2	U2	Tipos, variables, strings, listas, diccionarios, tuplas, archivos
3	U3	Condicionales, bucles, comprehensions, leer errores
4	U4	Funciones, parámetros, scope, lambda
5	U5	Clases, properties, herencia, dataclasses, Enum
6	U6	Módulos, PEP 8, excepciones, logging, pytest, Git
7	U7	NumPy, Series, DataFrames, filtros, nulos
8	U7 + U8	groupby, merge, pivot_table; Matplotlib y seaborn
9	U9 + U10	Bases de datos, APIs REST; estadística con SciPy
10	U11 + U12	Machine learning con scikit-learn; rentas, opciones, cópulas, Chain Ladder

Cómo trabajar cada día

Cada día tiene su carpeta (`dia_01_entorno_e_ia/` ... `dia_10_ml_y_proyecto/`) con un `README.md` y tres tipos de notebook, siempre por separado:

1. **Teoría** (`01_teoría*.ipynb` , `02_teoría*.ipynb` ...): explicación y código comentado. Ejecuta cada celda y lee lo que produce. En los bloques «Error típico», intenta **predecir qué va a fallar** antes de ejecutar.
2. **Ejercicios** (`NN_ejercicios.ipynb`): sin soluciones. Inténtalo en serio antes de pedir ayuda.
3. **Soluciones** (`NN_soluciones.ipynb`): ábrelas solo después de intentarlo. Compara tu enfoque con el propuesto; a menudo hay más de una forma correcta.

PARA LAS SESIONES

- **Prepara tu puesto:** si puedes, dos pantallas (la videollamada en una y VS Code en la otra); con una sola, divídela con `Windows + ←` y `Windows + →`. Auriculares con micrófono.
- **Antes de cada sesión:** ejecuta `git pull` en la carpeta del curso para recibir las novedades y las soluciones del día anterior, y abre Jupyter con doble clic en `abrir_jupyter.bat` (en Mac, `bash abrir_jupyter.sh`).
- **Antes de `git pull`**, guarda tu trabajo con un commit (`git add .` y `git commit -m "Mi trabajo"`) para que no se mezcle con las actualizaciones.
- **Cuando algo falle:** copia el mensaje de error completo **como texto** y pégalo en el chat, en vez de enviar una foto. Si hace falta, te pediremos que compartas pantalla.

LA REGLA DE ORO DE LOS NOTEBOOKS

Un notebook recuerda todo lo que has ejecutado, en el orden en que lo has hecho, aunque hayas borrado celdas. Cuando algo «no cuadra», usa **Kernel → Restart Kernel and Run All Cells** en Jupyter (en VS Code, **Reiniciar** y luego **Ejecutar todo**): vuelve a ejecutarlo todo desde cero, en orden. Si funciona así, funciona de verdad.

Cómo leer este manual

Cada unidad tiene los mismos bloques:

QUÉ VAS A APRENDER

Los objetivos de la unidad.

SI VIENES DE EXCEL

La equivalencia con lo que ya sabes hacer, y dónde se rompe la analogía.

PROBLEMA TÍPICO

Lo que te va a fallar, por qué, y cómo arreglarlo.

CON IA

Cómo usar Claude Code en esa unidad y qué verificar antes de dar por buena la respuesta.

Y termina con una **autoevaluación** de preguntas cortas, con respuestas en el apéndice E.

0.2 Tu entorno de trabajo

La guía paso a paso está en `docs/guia_instalacion.pdf` (y en `docs/guia_alumno_setup.md`). Si puedes, hazla antes de la primera sesión; si no, en la primera sesión veremos cómo hacerlo y tendrás que tenerlo listo para la segunda. Aquí tienes el resumen y los problemas más frecuentes.

OPCIÓN RÁPIDA

Con la carpeta del curso ya descargada, en **Windows** haz doble clic en `instalar_entorno.bat`; en **Mac**, abre Terminal, escribe `bash` y un espacio, arrastra `instalar_entorno.sh` a la ventana y pulsa Intro. Los dos instalan lo que falte y terminan con el checklist. Lo que salga como FALLA, hazlo a mano con la guía (que incluye una página para Mac).

PASO	QUÉ HACER	COMPROBACIÓN
1. Python	Instalar desde python.org (3.12 o superior; recomendada 3.13), marcando «Add python.exe to PATH» .	<code>python --version</code>
2. Entorno virtual	<code>python -m venv .venv</code> → <code>.venv\Scripts\activate</code> → <code>pip install -r requirements.txt</code>	Ves <code>(.venv)</code> al principio de la línea
3. Editor	VS Code con las extensiones Python y Jupyter, para los archivos <code>.py</code> y Git (sobre todo a partir del Día 6).	VS Code abre la carpeta del curso
4. Notebooks	Jupyter Notebook (recomendado): doble clic en <code>abrir_jupyter.bat</code> (Mac: <code>bash abrir_jupyter.sh</code>). Ya usa el entorno del curso, sin elegir kernel.	La primera celda muestra <code>[1]</code>
5. Git	Instalar desde git-scm.com y configurar <code>user.name</code> y <code>user.email</code> .	<code>git --version</code>
6. Claude Code	En PowerShell: <code>irm https://claude.ai/install.ps1 iex</code> . Abrir una terminal nueva y comprobar la versión. El inicio de sesión lo veremos en la primera sesión.	<code>claude --version</code>

```
# Checklist final, con el entorno activado:
python --version
git --version
claude --version
jupyter --version
python -c "import pandas, numpy, matplotlib; print('ok')"
```

PROBLEMAS TÍPICOS DE INSTALACIÓN

- **'python' no se reconoce como un comando**: no está en el PATH. Reinstala marcando la casilla.
- **python abre la Microsoft Store**: desactiva los alias en Configuración → Aplicaciones → Alias de ejecución de aplicaciones.
- **PowerShell no deja activar el entorno** (*la ejecución de scripts está deshabilitada*): ejecuta una vez `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`, o usa `cmd`.
- **ModuleNotFoundError en el notebook**: el kernel no es el de `.venv`. Cámbialo arriba a la derecha. Si no aparece: `python -m ipykernel install --user --name=curso-python-cac`.
- **Olvidar activar el entorno**: hay que activarlo cada vez que abras una terminal nueva.

0.3 Los datasets del curso

Todos están en `data/raw/`, son **sintéticos** (generados por los scripts de `data/generators/`) y no se editan a mano. Desde un notebook de un día, la ruta es relativa: `pd.read_csv("../data/raw/cartera_polizas.csv")`.

ARCHIVO	FILAS	COLUMNAS PRINCIPALES	SE USA EN
<code>cartera_polizas.csv</code>	3.000	<code>poliza_id</code> , <code>edad_tomador</code> , <code>producto</code> (AUTO, HOGAR, VIDA, SALUD), <code>provincia</code> , <code>canal_venta</code> , <code>fecha_alta</code> , <code>fecha_baja</code> , <code>prima_anual</code> , <code>franquicia</code> , <code>anulada</code>	Días 7-10
<code>siniestros.csv</code>	1.200	<code>siniestro_id</code> , <code>poliza_id</code> , <code>producto</code> , <code>fecha_siniestro</code> , <code>tipo_siniestro</code> , <code>importe_pagado</code> , <code>estado</code>	Días 7-9
<code>compras_seguros.csv</code>	815	Compras con <code>importe</code> , <code>descuento_aplicado_pct</code> , <code>satisfaccion</code> . Tiene nulos y duplicados a propósito .	Ejercicios del Día 7
<code>primas_mensuales.csv</code>	192	<code>anio</code> , <code>mes</code> , <code>producto</code> , <code>num_polizas</code> , <code>primas_emitidas</code>	Día 8 (heatmap)
<code>polizas_auto_anulacion.csv</code>	5.000	Conductor, vehículo, <code>forma_pago</code> , <code>num_siniestros_3_anios</code> , <code>recibo_impagado_alguna_vez</code> , <code>prima_anual</code> , <code>anulada</code>	Día 10 y proyecto final

Detalle curioso de `cartera_polizas.csv`: 2.629 de las 3.000 pólizas tienen `fecha_baja` vacía. No es un dato perdido: significa que la póliza sigue activa. Lo trabajarás el Día 7.

PARTE I

El temario, unidad por unidad

Las 12 unidades del programa en el orden del curso. Cada una resume lo esencial de sus notebooks de teoría, con el código que necesitas tener a mano y los problemas que te vas a encontrar.

U1 Entorno de trabajo e IA aplicada

Día 1 · `dia_01_entorno_e_ia/`

QUÉ VAS A APRENDER

- Moverte por la terminal de Windows y entender para qué sirve un entorno virtual.
- Ejecutar código en un notebook y en un script, y recuperarte cuando algo no cuadra.
- Reconocer la estructura del repositorio del curso.
- Usar Claude Code: el flujo básico, cómo escribir un buen prompt y cómo detectar una alucinación.

La terminal en cinco comandos

COMANDO	QUÉ HACE
<code>cd carpeta / cd ..</code>	Entrar en una carpeta / subir un nivel
<code>dir</code> (o <code>ls</code> en PowerShell)	Listar el contenido de la carpeta actual
<code>python script.py</code>	Ejecutar un script
<code>.venv\Scripts\activate</code>	Activar el entorno virtual del curso
<code>pip list</code>	Ver qué librerías hay instaladas en el entorno

Truco: pulsa **Tab** para autocompletar nombres de carpetas y **↑** para recuperar el último comando.

Notebook frente a script

	NOTEBOOK (<code>.IPYNB</code>)	SCRIPT (<code>.PY</code>)
Ejecución	Celda a celda, en el orden que tú elijas	Entero, de arriba abajo
Estado	El kernel recuerda todo lo ejecutado	Empieza de cero cada vez
Ideal para	Explorar datos, aprender, documentar un análisis	Código reutilizable, tests, procesos automáticos

Qué editor usar

Para los notebooks, el curso usa **Jupyter Notebook**: doble clic en `abrir_jupyter.bat` (en Mac, `bash abrir_jupyter.sh`) y se abre en el navegador, ya con el entorno del curso. **VS Code** reúne los archivos `.py`, la terminal y Git en una sola ventana, y es lo que necesitarás a partir del Día 6, cuando el código salga de los notebooks a módulos y tests; también abre notebooks, eligiendo el kernel `.venv`. Si ya trabajas con PyCharm, puedes seguir con él. **Anaconda no hace falta**: el curso usa un entorno virtual con versiones fijadas en `requirements.txt`.

PROBLEMA TÍPICO: «FUNCIONA EN MI NOTEBOOK, PERO NO AL REINICIAR»

Has ejecutado una celda que ya no existe, o las celdas en otro orden. El número `[7]` de cada celda indica en qué orden se ejecutó. **Solución**: Restart & Run All, y corregir lo que falle.

Claude Code

Claude Code es un asistente de programación que trabaja **dentro de tu proyecto**: puede leer los archivos, proponer cambios y ejecutar comandos (siempre pidiéndote permiso). Se arranca con `claude` desde la terminal, en la carpeta del curso. El archivo `CLAUDE.md` de la raíz del repositorio le da el contexto del proyecto: ábrelo, es un buen ejemplo de cómo explicarle a un asistente cómo trabajas.

Anatomía de un buen prompt de programación

PARTE	EJEMPLO
Contexto	«Tengo <code>data/raw/cartera_polizas.csv</code> con las columnas <code>producto</code> , <code>provincia</code> y <code>prima_anual</code> ...»
Objetivo	«...y quiero la prima media por producto y provincia...»
Restricciones	«...solo con pandas, sin bucles, compatible con pandas 2.3...»
Formato de salida	«...como una función con docstring y un ejemplo de uso.»

Y añade siempre, aunque sea para ti: **¿cómo voy a comprobar que el resultado es correcto?**

ALUCINACIONES

Un modelo de lenguaje genera el texto más plausible, no necesariamente el verdadero. Puede inventarse un método que «suena» a pandas pero no existe, un argumento que esa función no tiene o una cifra que no ha calculado. El ejercicio 4 del Día 1 te pondrá delante un caso real. **Cómo detectarlo**: ejecuta el código, consulta la documentación (`help(df.query)` o `df.query?` en Jupyter) y contrasta el resultado con un caso que sepas calcular.

AUTOEVALUACIÓN U1

1. Has definido la variable `tasa` en una celda y luego la has borrado. ¿Puedes seguir usando `tasa` en otras celdas? ¿Qué pasará si haces Restart & Run All?
2. ¿Por qué el curso usa un entorno virtual con un `requirements.txt` de versiones fijas?
3. Un asistente te propone `df.groupby("producto").prima_anual.media()`. ¿Qué sospechas, y cómo lo compruebas?

Notebooks: `01_teoría_entorno_windows` · `02_teoría_introducción_ia_claude_code` · `03_ejercicios` · `04_soluciones`

U2 Fundamentos de Python

Día 2 · `dia_02_fundamentos/`

QUÉ VAS A APRENDER

- Tipos numéricos, operadores y precedencia.
- Variables como referencias a objetos; nombres según PEP 8.
- Booleanos, comparaciones y la diferencia entre `==` e `is`.
- Strings, listas, diccionarios y tuplas; mutabilidad.
- Leer y escribir archivos de texto.

Números y operadores

```
prima = 350.0          # float
n_polizas = 3000       # int
prima * 1.08            # 378.0
7 / 2                  # 3.5 (división siempre da float)
7 // 2                 # 3 (división entera)
7 % 2                  # 1 (resto)
1.04 ** 10             # 1.4802... (potencia: factor de capitalización a 10 años)
-2 ** 2                # -4 (** va antes que el signo menos)
```

PROBLEMA TÍPICO: 0.1 + 0.2 NO ES 0.3

```
0.1 + 0.2 == 0.3       # False (da 0.30000000000000004)
import math
math.isclose(0.1 + 0.2, 0.3) # True
round(0.5), round(1.5)    # (0, 2): redondeo al par («del banquero»)
```

Los `float` se guardan en binario y 0,1 no tiene representación exacta, igual que 1/3 no la tiene en decimal. **Nunca compares floats con `==`**: usa `math.isclose`. Para importes que deben cuadrar al céntimo, existe `decimal.Decimal("0.10")`.

Variables: etiquetas, no cajas

```
primas = [320.5, 410.0]
alias = primas          # NO es una copia: dos nombres para la misma lista
alias.append(99)
print(primas)           # [320.5, 410.0, 99] (¡ha cambiado!)
copia = primas.copy()   # esto sí es una copia
```

Nombres según PEP 8: `snake_case` para variables y funciones (`prima_anual`), `MAYUSCULAS` para constantes (`TASA_IPS = 0.08`), sin acentos ni espacios, y descriptivos (`prima_media` mejor que `pm`).

Booleanos y comparaciones

```
edad = 45
18 <= edad < 65          # True (comparaciones encadenadas)
edad > 60 or prima > 1000 # and, or, not
x = None
x is None                 # True: 'is' solo para None, True y False
"AUTO" == "auto"          # False: distingue mayúsculas
```

Strings

```
poliza = "POL100000"
poliza[0:3]          # 'POL' (índices desde 0; el final no se incluye)
poliza[-3:]          # '000' (negativos: desde el final)
" auto ".strip().upper() # 'AUTO'
"AUTO;HOGAR".split(";") # ['AUTO', 'HOGAR']
f"Prima: {prima:,.2f} €" # 'Prima: 350.00 €'
f"Tasa: {0.035:.1%}"     # 'Tasa: 3.5%'
```

Los strings son **inmutables**: `poliza[0] = "X"` da `TypeError`. Los métodos devuelven un string nuevo, así que hay que guardarlo: `nombre = nombre.upper()`.

Listas, diccionarios y tuplas

```
primas = [455.69, 274.72, 812.30]
primas.append(390.0); len(primas); sum(primas); max(primas)
primas[1:]          # [274.72, 812.3, 390.0]

poliza = {"id": "POL100000", "producto": "AUTO", "prima": 455.69}
poliza["prima"]      # 455.69
poliza.get("franquicia", 0) # 0 (sin KeyError si no existe)
poliza["franquicia"] = 450 # añadir o modificar
for clave, valor in poliza.items(): ...

periodo = (2024, 1) # tupla: inmutable
anio, trimestre = periodo # unpacking
```

TIPO	¿SE PUEDE MODIFICAR?	ÚSALO PARA
<code>list</code> <code>[]</code>	Sí	Una colección que crece: las primas de una cartera
<code>dict</code> <code>{k: v}</code>	Sí	Buscar por clave: los datos de una póliza
<code>tuple</code> <code>()</code>	No	Un registro fijo: (año, trimestre)
<code>set</code> <code>{ }</code>	Sí	Valores únicos: las provincias distintas

SI VIENES DE EXCEL

Un diccionario es como un **BUSCARV** exacto: das la clave y devuelve el valor. Dónde se rompe la analogía: si repites una clave, **BUSCARV** devuelve la **primera** coincidencia, pero el diccionario se queda con la **última**, sin avisar: `{"P1": 100, "P1": 200}` es `{"P1": 200}`. Y una lista de diccionarios con las mismas claves es, en el fondo, una hoja de cálculo: filas y columnas. Esa idea vuelve el Día 7 con pandas.

Archivos de texto

```
with open("polizas_activas.txt", "r", encoding="utf-8") as f:
    for linea in f:
        print(linea.strip())

with open("resumen.txt", "w", encoding="utf-8") as f:    # "w" BORRA lo que hubiera
    f.write(f"Pólizas: {len(primas)}\n")
```

Modos: "r" leer, "w" escribir desde cero, "a" añadir al final. `with` cierra el archivo solo, aunque haya un error. En Windows, pon siempre `encoding="utf-8"` para que los acentos no salgan mal.

PROBLEMAS TÍPICOS

- `KeyError: 'prima'` : la clave no existe en el diccionario (¿mayúsculas? ¿una errata?). Usa `.get()` o comprueba con `"prima" in poliza`.
- `TypeError: can only concatenate str (not "float") to str` : `"Prima: " + 350.0`. Usa una f-string.
- `IndexError: list index out of range` : una lista de 3 elementos tiene índices 0, 1 y 2.
- Una ruta de Windows como `"C:\nuevo"` contiene un salto de línea (`\n`). Escribe `r"C:\nuevo"` o usa barras normales.

CON IA

Pide a Claude una analogía de Excel para un concepto de Python y, después, **busca dónde se rompe**. El notebook 05 lo hace con los diccionarios y BUSCARV: comprueba tú el caso de las claves repetidas.

AUTOEVALUACIÓN U2

1. ¿Qué imprime `a = [1, 2]; b = a; b.append(3); print(a)` ? ¿Y si la primera línea es `b = a.copy()` ?
2. ¿Qué devuelve `"POL100123"[3:6]` ?
3. ¿Por qué `(2024, 1)` puede ser clave de un diccionario y `[2024, 1]` no?
4. ¿Cómo escribirías 1234.5 como `'1,234.50'` con una f-string?

Notebooks: [01_teoría_numeros_variables](#) · [02_teoría_booleanos_operadores](#) · [03_teoría_strings](#) · [04_teoría_listas](#) · [05_teoría_diccionarios](#) · [06_teoría_tuplas](#) · [07_teoría_io_archivos](#) · [08_ejercicios](#) · [09_soluciones](#)

U3 Control de flujo

Día 3 · `dia_03_control_de_flujo/`

QUÉ VAS A APRENDER

- Tomar decisiones con `if / elif / else`.
- Repetir con `while` y `for`; controlar el bucle con `break`, `continue` y `else`.
- Recorrer listas, diccionarios y rangos con `enumerate` y `zip`.
- Crear listas y diccionarios en una línea con comprehensions.
- Leer un traceback y depurar tus primeros errores.

Condicionales

```
if siniestralidad > 0.8:
    recargo = 0.15
elif antigüedad >= 5:
    recargo = -0.05          # descuento por fidelidad
else:
    recargo = 0.0

nivel = "alto" if prima > 1000 else "normal"    # versión en una línea
```

La **indentación** (4 espacios) no es estética: define qué líneas están dentro de cada bloque. Solo se ejecuta la **primera** rama que se cumple, así que el orden de los `elif` importa.

En un `if` se consideran «falsos» `0`, `""`, `[]`, `{}` y `None`. Por eso `if primas:` significa «si la lista no está vacía».

Bucles

```
for prima in primas:                # recorrer una lista
    total += prima

for i in range(1, 11):               # 1, 2, ..., 10 (el 11 no se incluye)
    factor = 1.04 ** i

for i, prima in enumerate(primas, start=1): # con número de orden
    print(i, prima)

for poliza_id, prima in zip(ids, primas): # dos listas en paralelo
    ...

saldo = 10_000
while saldo > 0:                     # mientras se cumpla la condición
    saldo -= cuota
```

PALABRA	EFEECTO
<code>break</code>	Sale del bucle inmediatamente
<code>continue</code>	Salta a la siguiente iteración
<code>else</code> (tras un <code>for</code>)	Se ejecuta si el bucle terminó sin <code>break</code> : útil para «no encontrado»

PROBLEMA TÍPICO: EL BUCLE INFINITO

Un `while` cuya condición nunca deja de cumplirse (olvidaste actualizar `saldo`) cuelga el notebook.

Solución: botón de interrumpir el kernel (■) o **Kernel** → **Interrupt**. Pon siempre un límite de seguridad en bucles con riesgo: `while saldo > 0 and n < 1000: .`

Comprehensions

```
caras = [p for p in primas if p > 500]           # filtrar
con_ips = [round(p * 1.08, 2) for p in primas]   # transformar
etiquetas = ["alta" if p > 500 else "baja" for p in primas]
prima_por_id = {pol["id"]: pol["prima"] for pol in cartera}
```

Léelo así: «dame *esto* para cada *elemento* de la *lista*, si se cumple la *condición*». Si no cabe cómodamente en una línea, un `for` normal es más claro.

Leer un traceback

```
Traceback (most recent call last):
  Cell In[12], line 3
      2 for p in cartera:
----> 3     total += p["prima_anual"]
KeyError: 'prima_anual'
```

Léelo **de abajo arriba**. La última línea dice **qué** ha pasado (`KeyError` : esa clave no existe). La flecha dice **dónde**: celda 12, línea 3. Si hay muchas líneas intermedias de librerías, busca la última que sea de tu código.

ERROR	QUÉ SIGNIFICA NORMALMENTE
<code>SyntaxError</code>	Falta <code>:</code> , un paréntesis o unas comillas; o usas <code>=</code> en vez de <code>==</code> en un <code>if</code>
<code>IndentationError</code>	Sangría incorrecta o bloque vacío
<code>NameError</code>	Variable no definida: errata, o una celda que no has ejecutado
<code>TypeError</code>	Operación entre tipos incompatibles: <code>"100" + 5</code>
<code>KeyError</code> / <code>IndexError</code>	La clave no existe / el índice está fuera de rango
<code>ZeroDivisionError</code>	División por cero, por ejemplo un ratio de un grupo vacío
<code>ValueError</code>	Tipo correcto con valor inválido: <code>float("12,5")</code> (con coma)

LOS ERRORES MÁS PELIGROSOS NO TIENEN TRACEBACK

Un `elif` en mal orden, un `<` que debía ser `<=` o un acumulador que no se reinicia producen resultados **incorrectos sin ningún mensaje**. La defensa: prueba el código con un caso pequeño (3 pólizas) que puedas calcular a mano, y usa `print(f"{variable=}")` para ver valores intermedios.

CON IA

Cuando tengas un traceback, pide a Claude primero que te **explique** el error y después la corrección. Antes de aceptarla, comprueba que entiendes por qué funciona y que el resultado es correcto, no solo que ya no da error.

AUTOEVALUACIÓN U3

1. ¿Qué valores genera `range(2, 10, 3)` ?
2. Escribe una comprehension que, a partir de `primas` , devuelva solo las mayores de 300 incrementadas un 5 %.
3. Un traceback termina en `NameError: name 'cartera' is not defined` , pero tú la definiste. ¿Qué ha pasado probablemente?

Notebooks: [01_teoría_condicionales](#) · [02_teoría_bucles_while](#) · [03_teoría_bucles_for](#) · [04_teoría_comprehensions](#) · [05_teoría_errores_y_debugging](#) · [06_ejercicios](#) · [07_soluciones](#)

U4 Funciones

Día 4 · dia_04_funciones/

QUÉ VAS A APRENDER

- Definir funciones con parámetros posicionales, por nombre y con valor por defecto.
- Documentarlas con docstrings y anotaciones de tipo.
- Usar `*args` y `**kwargs`.
- Entender el alcance de las variables (regla LEGB).
- Escribir funciones `lambda` y usarlas con `map`, `filter` y `sorted`.
- Bonus: decoradores y `functools`.

Anatomía de una función

```
def prima_pura(frecuencia: float, severidad: float, recargo: float = 0.0) -> float:
    """Calcula la prima pura con un recargo opcional.

    Args:
        frecuencia: Siniestros esperados por año.
        severidad: Coste medio por siniestro, en euros.
        recargo: Recargo proporcional (0.1 = 10 %).

    Returns:
        Prima pura recargada, en euros.
    """
    return frecuencia * severidad * (1 + recargo)

prima_pura(0.08, 1500)                # 120.0
prima_pura(0.08, 1500, recargo=0.1)  # 132.0
prima_pura(severidad=1500, frecuencia=0.08)  # por nombre, en cualquier orden
```

- Los parámetros con valor por defecto van **después** de los obligatorios.
- Las anotaciones (`: float` , `-> float`) documentan, pero Python no las comprueba al ejecutar.
- Una función sin `return` devuelve `None`.
- Varios valores de retorno: `return media, desviacion` devuelve una tupla, que puedes desempaquetar con `m, d = f()`.

PROBLEMA TÍPICO: PRINT NO ES RETURN

Una función que hace `print(resultado)` muestra el valor pero devuelve `None` : `x = f()` deja `x` vacío. Las funciones de cálculo deben usar `return`.

*args y **kwargs

```
def total(*importes):                # acepta cualquier número de argumentos
    return sum(importes)
total(100, 250, 75)                 # 425

def crear_poliza(poliza_id, **datos): # argumentos por nombre libres
    return {"id": poliza_id, **datos}
crear_poliza("POL1", producto="AUTO", prima=455.69)

valores = [100, 250]; total(*valores) # * desempaqueta una lista
```

Alcance: la regla LEGB

Cuando usas un nombre, Python lo busca en este orden: **Local** (dentro de la función), **Enclosing** (la función que la contiene), **Global** (el notebook o el módulo) y **Built-in** (`len`, `sum` ...). Las variables creadas dentro de una función no existen fuera.

PROBLEMAS TÍPICOS

- **Pisar una función built-in:** `sum = 0` o `list = [...]` hacen que después `sum(primas)` dé `TypeError: 'int' object is not callable`. Cambia el nombre y reinicia el kernel.
- **Valor por defecto mutable:** `def f(x, lista=[])` comparte la misma lista entre todas las llamadas y va acumulando. Usa `lista=None` y dentro `if lista is None: lista = []`.
- `SyntaxError: non-default argument follows default argument`: has puesto un parámetro obligatorio detrás de uno con valor por defecto.

lambda, map, filter y sorted

```
sorted(cartera, key=lambda p: p["prima"], reverse=True) # ordenar por prima
list(map(lambda p: p * 1.08, primas))                  # = [p * 1.08 for p in primas]
list(filter(lambda p: p > 500, primas))                 # = [p for p in primas if p > 500]
```

Una `lambda` es una función anónima de una sola expresión. Donde más la usarás es como `key=` al ordenar y, más adelante, dentro de pandas.

Bonus: decoradores y functools

Un decorador (`@timer`) envuelve una función para añadirle comportamiento, como medir el tiempo o registrar llamadas. `functools.lru_cache` memoriza resultados y `functools.partial` fija argumentos. No son imprescindibles para el resto del curso.

CON IA: REFACTORIZAR CON RED DE SEGURIDAD

1. Antes de pedir el cambio, guarda el resultado actual de la función para varios casos, incluidos los casos borde (0, negativos, listas vacías).
2. Pide a Claude que la refactorice, la documente y te proponga casos borde.
3. Comprueba con `assert` que la nueva versión da exactamente los mismos resultados. Si no, no la aceptes.

AUTOEVALUACIÓN U4

1. ¿Qué devuelve `def f(x): x * 2` al llamar a `f(3)` ?
2. Escribe una función `valor_actual(capital, i, n)` que devuelva `capital / (1 + i) ** n`, con docstring y tipos.
3. ¿Por qué da error `def f(recargo=0.1, prima):` ?
4. Ordena una lista de diccionarios de pólizas por `"edad"` de menor a mayor.

Notebooks: [01_teoría_funciones_básicas](#) · [02_teoría_args_kwargs](#) · [03_teoría_scope_namespaces](#) · [04_teoría_lambda_map_filter](#) · [05_teoría_decoradores_bonus](#) · [06_teoría_functools_bonus](#) · [07_ejercicios](#) · [08_soluciones](#)

U5 Programación orientada a objetos

Día 5 · [dia_05_poo/](#)

QUÉ VAS A APRENDER

- Definir clases con atributos de instancia y de clase.
- Métodos de instancia, `@classmethod` y métodos especiales (`__repr__`, `__eq__`, `__lt__`, `__len__`).
- `@property` para atributos calculados o validados.
- Herencia con `super().__init__()`.
- `dataclasses` y `Enum`.
- Revisar código con un fallo sutil de lógica.

Clase e instancia

Una **clase** es el molde (la «póliza» en abstracto); una **instancia** es cada objeto concreto (la póliza POL100000). Cada instancia tiene sus datos (**atributos**) y sabe hacer cosas (**métodos**).

```
class Poliza:
    TASA_IPS = 0.08 # atributo de clase: común a todas

    def __init__(self, poliza_id: str, producto: str, prima: float):
        self.poliza_id = poliza_id # atributos de instancia
        self.producto = producto
        self._prima = prima

    @property
    def prima(self) -> float:
        return self._prima

    @prima.setter
    def prima(self, valor: float) -> None:
        if valor < 0:
            raise ValueError("La prima no puede ser negativa")
        self._prima = valor

    @property
    def prima_total(self) -> float: # se calcula cada vez: siempre al día
        return self._prima * (1 + self.TASA_IPS)

    @classmethod
    def desde_dict(cls, d: dict) -> "Poliza":
        return cls(d["poliza_id"], d["producto"], d["prima_anual"])

    def __repr__(self) -> str:
        return f"Poliza({self.poliza_id!r}, {self.producto!r}, {self._prima})"

p = Poliza("POL100000", "AUTO", 455.69)
p.prima_total # 492.1452 (sin paréntesis: es una property)
p.prima = -10 # ValueError
```

`self` es la propia instancia. Se escribe en la definición del método pero no al llamarlo: `p.prima_total` equivale internamente a pasar `p` como `self`.

Herencia

```
class PolizaAuto(Poliza):
    def __init__(self, poliza_id: str, prima: float, potencia_cv: int):
        super().__init__(poliza_id, "AUTO", prima)    # ¡no lo olvides!
        self.potencia_cv = potencia_cv
```

Una `PolizaAuto` **es una** `Poliza` : hereda todos sus métodos y puede añadir o redefinir los suyos.

Dataclasses y Enum

```
from dataclasses import dataclass, field
from enum import Enum
from datetime import date

class Producto(Enum):
    AUTO = "AUTO"
    HOGAR = "HOGAR"
    VIDA = "VIDA"
    SALUD = "SALUD"

@dataclass
class Siniestro:
    siniestro_id: str
    fecha: date
    importe_pagado: float
    producto: Producto
    pagos: list[float] = field(default_factory=list)    # nunca: pagos = []

s = Siniestro("SIN500001", date(2023, 7, 12), 2096.22, Producto.AUTO)
```

`@dataclass` escribe por ti el `__init__`, el `__repr__` y el `__eq__`. Un `Enum` limita los valores posibles: `Producto("DECESOS")` da error, mientras que un string mal escrito pasaría desapercibido.

PROBLEMAS TÍPICOS

- `TypeError: __init__() missing 1 required positional argument : faltan argumentos al crear el objeto, o has olvidado self en la definición.`
- `AttributeError: 'PolizaAuto' object has no attribute '_prima' : has olvidado super().__init__(...).`
- **Código que funciona pero calcula mal:** con clases aparecen fallos que no lanzan ninguna excepción. Encontrarlos es precisamente el objetivo del bloque de revisión crítica de este día.

CON IA, O MEJOR DICHO, SIN IA

El bloque de este día es distinto: se te presenta código que **funciona pero calcula mal**, y tienes que encontrar el fallo por tu cuenta. El ejercicio 4 **no se resuelve preguntando a un asistente**: el objetivo es entrenar tu criterio para revisar código, sea tuyo, de un compañero o de una IA.

AUTOEVALUACIÓN U5

1. Si cambias `Poliza.TASA_IPS = 0.10`, ¿qué pasa con `prima_total` de las pólizas que ya existen?
2. ¿Por qué `p.prima_total` se escribe sin paréntesis?
3. ¿Qué problema evita `field(default_factory=list)` ?

Notebooks: `01_teoría_clases_atributos_metodos` · `02_teoría_properties_herencia` · `03_teoría_dataclasses_enum` · `04_ejercicios` · `05_soluciones`

U6 Código profesional

Día 6 (puede ocupar dos jornadas) · `dia_06_codigo_profesional/`

QUÉ VAS A APRENDER

- Organizar el código en módulos y paquetes, y seguir el estilo PEP 8.
- Gestionar errores con `try / except / else / finally` y crear tus propias excepciones.
- Registrar lo que pasa con `logging`.
- Escribir tests con `pytest`.
- Trabajar con Git: commits, ramas, merge, revert y remotos.

Módulos y paquetes

```
tarificacion/          # paquete
├─ __init__.py         # lo convierte en paquete
├─ primas.py           # módulo
└─ excepciones.py
tests/
└─ test_primas.py
```

```
from tarificacion.primas import prima_pura    # importar una función
import tarificacion.primas as pr             # o el módulo con un alias

# Al final de primas.py:
if __name__ == "__main__":                   # solo se ejecuta con: python primas.py
    print(prima_pura(0.08, 1500)) # no al importar el módulo
```

PEP 8 en diez segundos

4 espacios de sangría; `snake_case` para funciones y variables; `PascalCase` para clases; `MAYUSCULAS` para constantes; líneas de unos 79-99 caracteres; espacios alrededor de `=` y de los operadores (pero no en `f(x=1)`); los imports al principio del archivo.

Excepciones

```
class ErrorTarificacion(Exception):
    """Error de negocio al calcular una tarifa."""

    try:
        tasa = float(texto)
    except ValueError:                # captura solo el error que esperas
        raise ErrorTarificacion(f"Tasa ilegible: {texto!r}")
    else:
        print("Tasa leída")           # solo si NO hubo error
    finally:
        print("Fin")                  # siempre
```

PROBLEMA TÍPICO: CAPTURARLO TODO

`except:` o `except Exception: pass` esconde **todos** los errores, incluidos tus propios fallos de programación. Captura siempre la excepción concreta que esperas.

Logging

```
import logging
logging.basicConfig(level=logging.INFO,
                    format="%asctime)s %(levelname)s %(message)s", force=True)
log = logging.getLogger(__name__)
log.info("Cargadas %d pólizas", 3000)
log.warning("%d pólizas sin fecha de alta", 12)
```

Niveles, de menor a mayor: `DEBUG`, `INFO`, `WARNING` (el que viene por defecto), `ERROR` y `CRITICAL`. Solo se muestran los mensajes de nivel igual o superior al configurado. `force=True` evita que, en un notebook, una segunda llamada a `basicConfig` se ignore sin avisar. A diferencia de `print`, el logging se puede filtrar, tiene fecha y hora, y puede guardarse en un archivo (`filename="proceso.log"`).

pytest

```
# tests/test_primas.py
import pytest
from tarificacion.primas import prima_pura

def test_prima_basica():
    assert prima_pura(0.08, 1500) == pytest.approx(120.0) # approx con floats

@pytest.mark.parametrize("frecuencia, severidad, esperado", [
    (0.08, 1500, 120.0),
    (0.0, 1500, 0.0),
])
def test_prima_varios(frecuencia, severidad, esperado):
    assert prima_pura(frecuencia, severidad) == pytest.approx(esperado)

def test_frecuencia_negativa():
    with pytest.raises(ValueError):
        prima_pura(-1, 100)
```

Se ejecuta con `pytest` (o `pytest -v`) desde la terminal. `pytest` encuentra solo los archivos `test_*.py` y las funciones `test_*`. Las **fixtures** (`@pytest.fixture`) preparan datos de prueba reutilizables; si están en `conftest.py`, cualquier test de la carpeta las recibe solo con poner su nombre como parámetro.

Git

Git guarda fotos (**commits**) de tu proyecto, cada una con un identificador (SHA), un autor, una fecha y un mensaje. Un cambio pasa por tres estados:

```
archivo modificado —git add—> preparado (staging) —git commit—> guardado en el historial
```

QUIERO...	COMANDO
Ver qué ha cambiado	<code>git status</code> , <code>git diff</code>
Guardar una versión	<code>git add archivo.py</code> → <code>git commit -m "Añade cálculo de prima pura"</code>
Ver el historial	<code>git log --oneline --graph</code>
Trabajar en una rama	<code>git switch -c feature/recargo ...</code> <code>git switch main</code> → <code>git merge feature/recargo</code>
Deshacer un commit ya compartido	<code>git revert SHA</code> (crea un commit que lo anula)
Traer un commit concreto de otra rama	<code>git cherry-pick SHA</code>
Sincronizar con GitHub	<code>git pull</code> , <code>git push</code> ; la revisión se hace con un Pull Request

`.gitignore` lista lo que **no** debe versionarse: `.env/` , `.env` (credenciales), `__pycache__`/ y datos confidenciales. Tienes más comandos en `docs/cheatsheet_git.md` .

CON IA

- **Tests generados por IA:** pide un test y luego **rompe la función a propósito** (cambia un `*` por un `+`). Si el test sigue pasando, no sirve. «El test pasa» no es garantía de nada.
- **Checklist antes de cada commit:** ¿lo he ejecutado de principio a fin? ¿pasan los tests? ¿he revisado el `git diff` ? ¿no hay credenciales ni datos reales? ¿el mensaje explica el porqué?

AUTOEVALUACIÓN U6

1. ¿En qué caso se ejecuta el bloque `else` de un `try` ?
2. ¿Por qué se usa `pytest.approx` y no `==` para comparar floats?
3. ¿Qué diferencia hay entre `git add` y `git commit` ?
4. ¿Por qué `.env` debe estar en `.gitignore` ?

Notebooks: `01_teoría_modulos_paquetes` · `02_teoría_excepciones` · `03_teoría_logging` · `04_teoría_pytest` · `05_teoría_git_github` · `06_ejercicios` · `07_soluciones`

U7 pandas y NumPy

Día 7 (fundamentos) y primera mitad del Día 8 (groupby, merge, pivot) · [dia_07_pandas_numpy/](#) ,
[dia_08_pandas_avanzado_visualizacion/](#)

QUÉ VAS A APRENDER

- Qué es un array de NumPy y por qué es tan rápido (vectorización, broadcasting).
- Series y DataFrames: crear, leer desde CSV o Excel, explorar.
- Seleccionar, crear y eliminar columnas; filtrar filas.
- Tratar nulos y duplicados con criterio de negocio.
- Agrupar (`groupby`), unir tablas (`merge` , `concat`) y hacer tablas dinámicas (`pivot_table`).
- Exportar resultados a CSV y Excel.

NumPy: operar con arrays enteros

```
import numpy as np
primas = np.array([455.69, 274.72, 812.30, 390.00])
primas * 1.08 # multiplica cada elemento, sin bucle
primas.mean(), primas.std()
primas[primas > 400] # filtrar con una condición
t = np.arange(1, 11) # 1..10
descuento = 1.04 ** -t # 10 factores de descuento de una vez
```

Vectorizar es escribir la operación sobre el array completo en vez de hacer un bucle. Es más corto y hasta 100 veces más rápido.

Cargar y explorar

```
import pandas as pd
cartera = pd.read_csv("../data/raw/cartera_polizas.csv",
                      parse_dates=["fecha_alta", "fecha_baja"])
cartera.shape # (3000, 12)
cartera.head() # primeras 5 filas
cartera.info() # columnas, tipos y nulos
cartera.describe() # estadísticos de las numéricas
cartera.producto.value_counts() # AUTO 1361, HOGAR 895, VIDA 429, SALUD 315
cartera.isna().sum() # nulos por columna
```

Una **Series** es una columna: valores más un índice. Un **DataFrame** es una tabla: varias Series que comparten el índice. `cartera["prima_anual"]` (o `cartera.prima_anual`) es una Series.

Seleccionar y filtrar

```
cartera[["poliza_id", "producto", "prima_anual"]] # varias columnas: doble corchete
cartera.loc[0:4, ["poliza_id", "prima_anual"]] # loc: por etiqueta (incluye el 4)
cartera.iloc[0:5, 0:3] # iloc: por posición (excluye el 5)

auto = cartera[cartera.producto == "AUTO"]
caras_madrid = cartera[(cartera.provincia == "Madrid") & (cartera.prima_anual > 500)]
cartera[cartera.producto.isin(["AUTO", "HOGAR"])]
cartera[cartera.edad_tomador.between(25, 40)] # incluye 25 y 40
cartera.query("producto == 'AUTO' and prima_anual > 500")
activas = cartera[cartera.fecha_baja.isna()] # 2.629 pólizas activas
```

Crear y modificar columnas

```
cartera["prima_con_ips"] = cartera.prima_anual * 1.08
cartera["anio_alta"] = cartera.fecha_alta.dt.year
cartera["tramo_edad"] = pd.cut(cartera.edad_tomador, bins=[17, 30, 45, 65, 100],
                              labels=["18-30", "31-45", "46-65", "65+"])
cartera["activa"] = cartera.fecha_baja.isna()
cartera = cartera.drop(columns=["nombre_asegurado"])
cartera = cartera.rename(columns={"prima_anual": "prima"})
```

Nulos y duplicados: primero entender, después decidir

HERRAMIENTA	QUÉ HACE	CUÁNDO TIENE SENTIDO
<code>isna()</code> / <code>notna()</code>	Detecta los nulos	Siempre, antes de decidir nada
<code>dropna(subset=[...])</code>	Elimina filas con nulos en esas columnas	Cuando el nulo es un error y hay pocos casos
<code>fillna(valor)</code>	Sustituye el nulo	Cuando el nulo tiene un significado: «sin siniestros» = 0
<code>duplicated(subset=[...])</code>	Marca las filas repetidas	Para contarlas e inspeccionarlas
<code>drop_duplicates(subset=[...], keep="first")</code>	Elimina las repetidas	Cuando sabes cuál es la copia buena

NUNCA UN DROPNA() AUTOMÁTICO

En `cartera_polizas.csv`, `cartera.dropna()` eliminaría las 2.629 pólizas activas, porque su `fecha_baja` está vacía. Un nulo puede significar «dato perdido», «no aplica» o «todavía no ha pasado». Esa decisión es de negocio, no de pandas.

groupby: agrupar y resumir

```
resumen = (cartera
            .groupby("producto")
            .agg(n_polizas=("poliza_id", "count"),
                 prima_total=("prima_anual", "sum"),
                 prima_media=("prima_anual", "mean"),
                 tasa_anulacion=("anulada", "mean"))
            .sort_values("prima_total", ascending=False))

cartera.groupby(["producto", "provincia"]).prima_anual.mean()      # varias claves
cartera["cuota_en_producto"] = (cartera.prima_anual /
                                cartera.groupby("producto").prima_anual.transform("sum"))  # misma forma que el original
```

`agg` reduce cada grupo a una fila; `transform` devuelve un valor por cada fila original, útil para añadirlo como columna. La media de una columna booleana (`anulada`) es la proporción de `True` .

merge: el BUSCARV de pandas

```
siniestros = pd.read_csv("../data/raw/siniestros.csv")

# 1) Agregar primero los siniestros por póliza (una póliza puede tener varios)
sin_poliza = siniestros.groupby("poliza_id", as_index=False).importe_pagado.sum()

# 2) Unir a la cartera, conservando todas las pólizas
base = cartera.merge(sin_poliza, on="poliza_id", how="left",
                    validate="one_to_one", indicator=True)
base["importe_pagado"] = base.importe_pagado.fillna(0)  # sin siniestros = 0

# 3) Ratio de siniestralidad por producto: cociente de SUMAS
g = base.groupby("producto")[["importe_pagado", "prima_anual"]].sum()
g["ratio"] = g.importe_pagado / g.prima_anual
```

HOW=	CONSERVA
"inner"	Solo las filas cuya clave está en las dos tablas
"left"	Todas las de la tabla izquierda (NaN donde no hay coincidencia)
"outer"	Todas las de las dos tablas

PROBLEMAS TÍPICOS DEL MERGE

- **Filas duplicadas:** si unes la cartera con los siniestros sin agregarlos antes, una póliza con 3 siniestros aparece 3 veces y su prima se suma 3 veces. `validate="one_to_one"` lanza un error si eso ocurre.
- **Ratio mal calculado:** el ratio de un grupo es Σ siniestros / Σ primas, no la media de los ratios de cada póliza.
- **Claves que no casan:** espacios o mayúsculas distintas (`"POL1 "` frente a `"POL1"`). Revisa `base._merge.value_counts()` .

pivot_table, concat y exportar

```
primas_mes = pd.read_csv("../data/raw/primas_mensuales.csv")
tabla = pd.pivot_table(primas_mes, index="anio", columns="mes",
                        values="primas_emitidas", aggfunc="sum", fill_value=0)

total = pd.concat([cartera_2023, cartera_2024], ignore_index=True) # apilar tablas

resumen.to_csv("resumen_productos.csv", index=False)
resumen.to_excel("resumen_productos.xlsx", sheet_name="productos")
# CSV que Excel en español abre bien:
resumen.to_csv("resumen.csv", sep=";", decimal=".", encoding="utf-8-sig")
```

PIVOT_TABLE PROMEDIA POR DEFECTO

La tabla dinámica de Excel suma; `pivot_table` calcula la **media** si no le dices otra cosa. Escribe siempre `aggfunc="sum"` (o la función que quieras).

SI VIENES DE EXCEL

EXCEL	PANDAS
Autofiltro	<code>df[df.producto == "AUTO"]</code>
SUMAR.SI	<code>df.loc[df.producto == "AUTO", "prima_anual"].sum()</code>
CONTAR.SI	<code>(df.producto == "AUTO").sum()</code>
PROMEDIO.SI	<code>df.loc[df.producto == "AUTO", "prima_anual"].mean()</code>
BUSCARV	<code>df.merge(otra, on="clave", how="left")</code>
Tabla dinámica	<code>pd.pivot_table(..., aggfunc="sum")</code>
SI()	<code>np.where(df.prima_anual > 500, "alta", "normal")</code>

OTROS PROBLEMAS TÍPICOS

- `ValueError: The truth value of a Series is ambiguous` : has usado `and` / `or` . Con pandas se usan `&` / `|` / `~`, y cada condición entre paréntesis.
- `SettingWithCopyWarning` : modificas un trozo filtrado (`auto = cartera[...]; auto["x"] = 1`). Usa `.copy()` al filtrar, o `cartera.loc[condición, "x"] = 1`.
- `KeyError: 'prima'` al seleccionar una columna: revisa `cartera.columns` (¿mayúsculas? ¿espacios?).
- Sumar dos Series con índices distintos da `NaN` : pandas alinea por etiqueta de índice, no por posición.

CON IA: TRADUCIR DE EXCEL Y VALIDAR

Pide a Claude la traducción a pandas de una fórmula de Excel que uses. Después, **valida contra un caso de control**: coge 5 filas, calcula el resultado a mano (o en Excel) y comprueba que el código da lo mismo para esas 5. Si no cuadra, el código está mal aunque se ejecute sin errores.

AUTOEVALUACIÓN U7

1. ¿Qué diferencia hay entre `cartera.loc[0:3]` y `cartera.iloc[0:3]` ?
2. Escribe el filtro de las pólizas de HOGAR con prima mayor que 300 y todavía activas.
3. ¿Cómo obtienes la tasa de anulación por canal de venta?
4. Después de un `merge` la tabla tiene más filas que la cartera original. ¿Qué ha pasado?

Notebooks Día 7: [01_teoría_numpy](#) · [02_teoría_series](#) · [03_teoría_dataframes_basico](#) ·
[04_teoría_seleccion_filtrado_nulos](#) · [05_ejercicios](#) · [06_soluciones](#) . Día 8: [01_teoría_groupby_avanzado](#) ·
[02_teoría_merge_concat_pivot_io](#)

U8 Visualización

Día 8 · `dia_08_pandas_avanzado_visualizacion/` 03-07

QUÉ VAS A APRENDER

- Cómo se organiza un gráfico en Matplotlib (Figure, Axes, Axis).
- Crear gráficos con `fig, ax = plt.subplots()` y combinar varios en una figura.
- Gráficos de distribución y de categorías con seaborn.
- Mapas de calor de correlaciones y de series año × mes.
- Revisar con criterio un gráfico, también uno generado por IA.

La plantilla que usarás siempre

```
import matplotlib.pyplot as plt
import seaborn as sns

fig, ax = plt.subplots(figsize=(8, 4))          # Figure = lienzo, Axes = gráfico
sns.histplot(data=siniestros, x="importe_pagado", bins=40, ax=ax)
ax.set(title="Coste de los siniestros", xlabel="Importe pagado (€)", ylabel="Nº de siniestros")
fig.tight_layout()
fig.savefig("hist_siniestros.png", dpi=150)
```

Varios gráficos a la vez: `fig, axes = plt.subplots(1, 2, figsize=(12, 4))` y luego `ax=axes[0]`, `ax=axes[1]`.

Qué gráfico elegir

QUIERO VER...	GRÁFICO	CÓDIGO
Cómo se distribuye una variable	Histograma, densidad	<code>sns.histplot(data=df, x="importe_pagado")</code> , <code>sns.kdeplot</code>
Una variable numérica por categoría	Caja o violín	<code>sns.boxplot(data=df, x="producto", y="importe_pagado")</code>
Cuántos hay de cada categoría	Barras de recuento	<code>sns.countplot(data=cartera, x="producto")</code>
Un promedio por categoría	Barras	<code>sns.barplot(data=cartera, x="producto", y="prima_anual")</code>
La relación entre dos numéricas	Dispersión	<code>sns.scatterplot(data=cartera, x="edad_tomador", y="prima_anual")</code> , <code>sns.jointplot</code>
Muchas relaciones a la vez	Matriz de dispersión	<code>sns.pairplot(cartera[cols])</code>
Correlaciones	Mapa de calor	<code>sns.heatmap(df[cols].corr(), annot=True, fmt=".2f", cmap="vlag", center=0)</code>
Estacionalidad año × mes	Mapa de calor	<code>pivot_table</code> + <code>sns.heatmap(tabla)</code>

PROBLEMAS TÍPICOS

- **barplot** muestra la media, no la suma, con una barra de error de confianza. Para totales: `estimator="sum"` , o agrega antes con `groupby` .
- **Mezclar plt. y ax.** : con varios subgráficos, `plt.title()` puede ir a parar al que no es. Usa siempre `ax.set_title()` o `ax.set(...)` .
- **displot , catplot , jointplot y pairplot** crean su propia figura y no aceptan `ax=` . Dentro de `subplots` , usa `histplot` , `boxplot` ...
- **Demasiadas figuras abiertas** consumen memoria. `plt.close(fig)` al terminar, si generas muchas en un bucle.
- **Parámetros antiguos**: en seaborn 0.13 es `violinplot(..., density_norm="count")` , no `scale="count"` . Mucho código de internet (y de IA) usa la versión antigua.

CON IA: CHECKLIST PARA REVISAR UN GRÁFICO

1. **Eje Y**: ¿empieza en cero cuando comparas magnitudes con barras?
2. **Paleta**: ¿distingue bien las categorías? ¿se lee en blanco y negro o con daltonismo?
3. **Tipo**: ¿responde a la pregunta? (distribución → histograma o caja; categorías → barras; tiempo → línea o heatmap)
4. **Dato**: ¿muestra la media o la suma? ¿de qué subconjunto?
5. **Etiquetas**: título, ejes y unidades.

Un número se valida con un `assert` ; un gráfico, con criterio.

AUTOEVALUACIÓN U8

1. ¿Qué es `ax` en `fig, ax = plt.subplots()` ?
2. ¿Qué gráfico usarías para comparar la distribución del importe pagado entre los cuatro productos?
3. Te dan un gráfico de barras de «primas por provincia» hecho con `sns.barplot` . ¿Qué dudas te plantea?

Notebooks: [03_teoría_matplotlib_basico](#) · [04_teoría_seaborn_distribucion_categorica](#) · [05_teoría_matrix_plots_heatmap](#) · [06_ejercicios](#) · [07_soluciones](#)

U9 Fuentes de datos: bases de datos y APIs

Día 9, mañana · `dia_09_datos_y_estadistica/` 01-03

QUÉ VAS A APRENDER

- Qué es una base de datos relacional: tablas, clave primaria y clave foránea.
- Qué es la SQL injection y cómo se evita.
- Crear, leer, actualizar y borrar registros (CRUD) con SQLAlchemy.
- Consumir una API REST con `requests` y gestionar sus errores.
- Guardar credenciales en variables de entorno, nunca en el código.

El modelo relacional

La tabla `polizas` tiene una **clave primaria** (`poliza_id`) que identifica cada fila de forma única. La tabla `sinistros` tiene una **clave foránea** `poliza_id` que apunta a la póliza correspondiente. Es la misma relación que usas en el `merge` de pandas, pero garantizada por la base de datos.

SQL injection

```
poliza_id = '' OR '1'='1' # texto introducido por alguien
sql = f"SELECT * FROM polizas WHERE poliza_id = '{poliza_id}'" # ¡MAL!
# Resultado: ... WHERE poliza_id = '' OR '1'='1' → devuelve TODAS las pólizas
```

Nunca construyas SQL pegando texto. Un ORM como SQLAlchemy envía los valores por separado de la consulta, así que no pueden alterarla.

CRUD con SQLAlchemy

```
from sqlalchemy import SQLModel, Field, Session, create_engine, select
from sqlalchemy.pool import StaticPool

class Poliza(SQLModel, table=True):          # una clase = una tabla
    poliza_id: str = Field(primary_key=True)
    producto: str
    provincia: str
    prima_anual: float

engine = create_engine("sqlite://",          # base de datos en memoria
                      connect_args={"check_same_thread": False},
                      poolclass=StaticPool)
SQLModel.metadata.create_all(engine)         # crea las tablas

with Session(engine) as session:
    # Create
    session.add(Poliza(poliza_id="POL100000", producto="AUTO",
                      provincia="Alicante", prima_anual=455.69))
    session.commit()
    # Read
    caras = session.exec(select(Poliza).where(Poliza.prima_anual > 400)).all()
    p = session.get(Poliza, "POL100000")
    # Update
    p.prima_anual = 470.0
    session.add(p); session.commit()
    # Delete
    session.delete(p); session.commit()
```

Hasta el `commit()` no se guarda nada. Para pasar a pandas: `pd.read_sql(select(Poliza), engine)`.

PROBLEMA TÍPICO: «LAS TABLAS DESAPARECEN»

Con SQLite en memoria (`"sqlite://"`), cada conexión nueva crea una base de datos vacía distinta. `poolclass=StaticPool` obliga a reutilizar siempre la misma conexión. Es justo el bloque de IA de la mañana: entender este parámetro con la documentación y comprobarlo quitándolo.

APIs REST

VERBO	USO	EQUIVALE A
GET	Leer un recurso	Read
POST	Crear	Create
PUT / PATCH	Reemplazar / modificar	Update
DELETE	Borrar	Delete

```

import os, requests
from dotenv import load_dotenv

load_dotenv() # lee el archivo .env
token = os.getenv("API_TOKEN")
respuesta = requests.get(f"{url_base}/polizas/POL100000",
                        headers={"Authorization": f"Bearer {token}"},
                        timeout=10) # pon siempre timeout
if respuesta.status_code == 404:
    print("La póliza no existe")
respuesta.raise_for_status() # lanza un error si es 4xx o 5xx
datos = respuesta.json() # JSON → dict de Python

```

CÓDIGO	SIGNIFICADO
200 / 201	Correcto / creado
400	La petición está mal formada
401	No autenticado: falta el token o no es válido
403	Autenticado, pero sin permiso
404	El recurso no existe
500	Error del servidor (no es culpa tuya)

CREDENCIALES

Nunca escribas un token o una contraseña en el código, en un notebook ni en un prompt. Van en un archivo `.env` que está en `.gitignore`. En el repositorio solo se versiona `.env.example`, con valores de mentira.

AUTOEVALUACIÓN U9

1. ¿Qué papel tiene `poliza_id` en la tabla de siniestros?
2. Has hecho `session.add(p)` pero no ves el cambio en la base de datos. ¿Qué te falta?
3. Una API devuelve 401. ¿Qué revisas primero?

Notebooks: `01_teoría_bdd_fundamentos` · `02_teoría_bdd_crud` · `03_teoría_apis_rest`

U10 Estadística aplicada con SciPy

Día 9, tarde · `dia_09_datos_y_estadistica/` 04-08

La estadística ya la conoces. Esta unidad trata de **cómo se calcula en Python** y de las trampas de la librería.

QUÉ VAS A APRENDER

- Medidas de tendencia central, dispersión, forma y correlación con `scipy.stats`.
- La API común de las distribuciones: `pdf` / `pmf`, `cdf`, `ppf`, `rvs`, `fit`.
- Normal, binomial y Poisson aplicadas a la cartera.
- Contrastes t y chi-cuadrado, e interpretar el p-valor.

Descriptiva

```
from scipy import stats
import numpy as np
x = cartera.prima_anual
np.mean(x), np.median(x), stats.mode(x.round(0))
np.average(x, weights=exposicion)          # media ponderada
stats.gmean(factores_crecimiento)          # media geométrica (tasas compuestas)
stats.hmean(x)                             # media armónica
np.percentile(x, [25, 50, 75, 99.5])
stats.skew(x), stats.kurtosis(x)            # kurtosis = EXCESO (la normal da 0)
stats.pearsonr(cartera.edad_tomador, cartera.prima_anual) # .statistic, .pvalue
stats.spearmanr(cartera.edad_tomador, cartera.prima_anual)
```

VARIANZA: PANDAS Y NUMPY NO COINCIDEN

`x.var()` de pandas usa $n - 1$ (muestral); `np.var(x)` usa n (poblacional). Para igualarlas: `np.var(x, ddof=1)`.

Distribuciones: una API para todas

```
edad = stats.norm(loc=45, scale=12)         # scale = DESVIACIÓN TÍPICA, no varianza
edad.pdf(45)                                # densidad en 45
edad.cdf(65)                                #  $P(X \leq 65)$ 
edad.sf(65)                                 #  $P(X > 65) = 1 - \text{cdf} \rightarrow 0.0478$ 
edad.ppf(0.995)                             # percentil 99,5 (inversa de la cdf)
edad.rvs(size=1000, random_state=42)        # simular

anuladas = stats.binom(n=20, p=0.12)         # anulaciones en 20 pólizas
anuladas.pmf(3)                             #  $P(\text{exactamente } 3)$ ; en discretas es pmf, no pdf
n_siniestros = stats.poisson(mu=0.8)
n_siniestros.pmf(0)                         #  $P(\text{ningún siniestro}) = e^{-0.8} \approx 0.449$ 

loc, scale = stats.norm.fit(cartera.edad_tomador) # estimación por máxima verosimilitud
```

MÉTODO	DEVUELVE
<code>pdf(x)</code> / <code>pmf(k)</code>	Densidad (continua) / probabilidad puntual (discreta)
<code>cdf(x)</code>	$P(X \leq x)$
<code>sf(x)</code>	$P(X > x)$
<code>ppf(q)</code>	El valor x con $P(X \leq x) = q$ (cuantil, VaR)
<code>rvs(size)</code>	Muestras aleatorias
<code>fit(datos)</code>	Parámetros estimados (solo distribuciones continuas)

Contrastes de hipótesis

```
stats.ttest_1samp(primas_auto, popmean=450)          # ¿la media es 450?
stats.ttest_ind(primas_madrid, primas_barcelona, equal_var=False) # Welch
tabla = pd.crosstab(cartera.canal_venta, cartera.anulada)
res = stats.chi2_contingency(tabla)                  # ¿canal y anulación son
independientes?
res.statistic, res.pvalue, res.dof, res.expected_freq
```

EL P-VALOR, BIEN LEÍDO

- Es la probabilidad de obtener un resultado al menos tan extremo como el observado si H_0 fuera cierta.
- **No** es la probabilidad de que H_0 sea cierta.
- No rechazar H_0 no demuestra que sea cierta.
- Con muchos datos, diferencias sin importancia práctica salen «significativas»: mira también el tamaño de la diferencia.

CON IA: DE LA ESTADÍSTICA AL NEGOCIO

Pide a Claude que traduzca el resultado de un test a un párrafo para dirección. Después, **verifica cifra a cifra** que el p-valor, las medias y los tamaños de muestra que cita coinciden con los que ha calculado Python. Los modelos de lenguaje redondean, confunden cifras o se inventan números con total seguridad.

AUTOEVALUACIÓN U10

1. ¿Con qué método de `stats.norm(45, 12)` calculas la edad que solo supera el 5 % de los tomadores?
2. ¿Por qué `stats.poisson(0.8).pdf(1)` da error?
3. ¿Qué test usarías para ver si la tasa de anulación depende de la forma de pago?

Notebooks: [04_teoría_estadística_descriptiva](#) · [05_teoría_distribuciones_probabilidad](#) · [06_teoría_pruebas_hipotesis](#) · [07_ejercicios](#) · [08_soluciones](#)

U11 Machine learning aplicado

Día 10, mañana · dia_10_ml_y_proyecto/ 01-03

QUÉ VAS A APRENDER

- Ajustar y evaluar regresiones con scikit-learn (`LinearRegression` , `PolynomialFeatures` , R^2).
- Construir un `Pipeline` con `ColumnTransformer` para el preprocesado.
- Clasificar con `LogisticRegression` y `RandomForestClassifier` .
- Interpretar precision, recall y la matriz de confusión en términos de negocio.
- Reconocer los cuatro riesgos: sobreajuste, fuga de datos, variables mal definidas y sesgos.

La receta de scikit-learn

Todos los modelos funcionan igual: `modelo.fit(X_train, y_train)` para aprender, `modelo.predict(X_test)` para predecir y `modelo.predict_proba(X_test)` para obtener probabilidades. `X` son las variables explicativas (un `DataFrame`) e `y` la variable objetivo.

Un modelo de anulación completo

```
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix

df = pd.read_csv("../data/raw/polizas_auto_anulacion.csv")
X = df.drop(columns=["poliza_id", "anulada"])
y = df["anulada"]
numericas = ["edad_conductor", "antiguedad_carnet", "antiguedad_vehiculo", "potencia_cv",
             "km_anuales", "num_siniestros_3_anios", "antiguedad_poliza_meses", "prima_anual"]
categoricas = ["tipo_vehiculo", "uso", "provincia", "canal_venta", "forma_pago",
               "recibo_impagado_alguna_vez"]

# 1) Separar ANTES de nada
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, stratify=y, random_state=42)

# 2) Preprocesado dentro del pipeline
preproceso = ColumnTransformer([
    ("num", Pipeline([("imputar", SimpleImputer(strategy="median")),
                     ("escalar", StandardScaler())]), numericas),
    ("cat", OneHotEncoder(handle_unknown="ignore"), categoricas),
])
modelo = Pipeline([("prep", preproceso),
                   ("clf", LogisticRegression(max_iter=1000, class_weight="balanced"))])

# 3) Entrenar solo con train, evaluar con test
modelo.fit(X_train, y_train)
y_pred = modelo.predict(X_test)
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

Adapta las listas de columnas a lo que decidas tras el EDA. `potencia_cv` tiene nulos a propósito, de ahí el `SimpleImputer`.

Métricas en lenguaje de negocio

	PREDICE: SE ANULA	PREDICE: SE MANTIENE
Real: se anula	VP · se contacta al cliente a tiempo	FN · baja que no vimos venir
Real: se mantiene	FP · llamada de retención innecesaria	VN · sin acción, sin coste

- **Precision** = $VP / (VP + FP)$: de las pólizas que señalo, cuántas se anulan de verdad.
- **Recall** = $VP / (VP + FN)$: de las que se anulan, cuántas detecto.
- **Accuracy** engaña cuando las clases están desbalanceadas: en `polizas_auto_anulacion.csv` se anula el 11,5 %, así que decir siempre «no se anula» acierta el 88,5 % sin detectar ni una sola anulación.
- `confusion_matrix` de scikit-learn pone la realidad en las filas y la predicción en las columnas: `[[VN, FP], [FN, VP]]`.
- El umbral por defecto es 0,5. Puedes elegir otro según el coste de cada error:
`(modelo.predict_proba(X_test)[: , 1] >= 0.3)`.

Regresión y sobreajuste

R^2 mide la proporción de varianza explicada. Si el R^2 de train es muy alto y el de test mucho más bajo, el modelo ha **memorizado** en vez de aprender: es sobreajuste. Con `PolynomialFeatures` de grado alto se ve muy claramente. Para elegir hiperparámetros sin mirar test existe `RandomizedSearchCV`, con validación cruzada sobre train; dentro de un pipeline, los parámetros se nombran `"clf__C"` (paso + doble guion bajo + parámetro).

LOS CUATRO RIESGOS

- **Sobreajuste**: muy bueno en train, malo en test.
- **Fuga de datos (*leakage*)**: usar información que no tendrías al predecir. Por ejemplo, escalar con todo el dataset antes del split, o incluir una variable que solo se conoce después de la anulación. **No da ningún error**: por eso todo `fit` va dentro del pipeline y solo con train.
- **Variables mal definidas**: ¿qué cuenta exactamente como «anulada»? ¿en qué ventana de tiempo?
- **Sesgos**: variables que actúan como proxy de características protegidas (por ejemplo, la provincia como proxy de renta).

CON IA

Pide a Claude que te explique cada paso de tu pipeline. Después, pásale un código con leakage escondido y comprueba si lo encuentra. Tú tienes que encontrarlo también: no des por buena su revisión sin entenderla.

AUTOEVALUACIÓN U11

1. ¿Por qué se usa `stratify=y` en `train_test_split` ?
2. ¿Qué pasa si en test aparece una provincia que no estaba en train y no has puesto `handle_unknown="ignore"` ?
3. El negocio prefiere no perder ninguna anulación aunque haga llamadas de más. ¿Qué métrica priorizas y hacia dónde mueves el umbral?

Notebooks: `01_teoría_regresión_lineal` · `02_teoría_pipelines_riesgos_ml` · `03_ejemplo_aplicado_prudencial`

U12 Cálculo actuarial-financiero en Python

Día 10, tarde · dia_10_ml_y_proyecto/ 04-09

Los conceptos los dominas. Aquí los implementarás con NumPy, SciPy y pandas, sin librerías especializadas, y **validarás cada resultado contra un caso de control conocido**. Ese es el hábito que la unidad quiere dejarte.

Rentas

```
def valor_actual_renta(pago: float, i: float, n: int, g: float = 0.0) -> float:
    """Valor actual de una renta pospagable de n pagos que crecen a tasa g."""
    t = np.arange(1, n + 1)
    return float((pago * (1 + g) ** (t - 1) / (1 + i) ** t).sum())

va = valor_actual_renta(250_000, 0.04, 10) # ≈ 2.027.724
assert np.isclose(va, 250_000 * (1 - 1.04 ** -10) / 0.04) # fórmula cerrada
assert np.isclose(valor_actual_renta(1, 0.04, 5_000), 1 / 0.04) # ≈ perpetua
valor_actual_renta(250_000, 0.04, 10, g=-0.03) # con caída de cartera ≈ 1.792.224
```

Opción europea por Monte Carlo

```
S0, K, r, sigma, T = 100.0, 105.0, 0.03, 0.20, 1.0
rng = np.random.default_rng(42)
z = rng.standard_normal(200_000)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * np.sqrt(T) * z)
pagos = np.exp(-r * T) * np.maximum(ST - K, 0) # np.maximum, NO max()
precio_mc = pagos.mean()
error = pagos.std(ddof=1) / np.sqrt(len(pagos))
# Control: Black-Scholes da 7,128 → |precio_mc - 7,128| < 4 * error
```

Cópula gaussiana

```
rho = 0.5
z = rng.multivariate_normal([0, 0], [[1, rho], [rho, 1]], size=50_000)
u = stats.norm.cdf(z) # uniformes con dependencia
auto = stats.lognorm(s=0.8, scale=np.exp(7)).ppf(u[:, 0]) # marginales (ilustrativas)
hogar = stats.lognorm(s=1.0, scale=np.exp(6.5)).ppf(u[:, 1])
np.percentile(auto + hogar, 99.5) # percentil de solvencia
# Control: Spearman(auto, hogar) ≈ 6/π · arcsen(ρ/2) = 0,4826
```

Chain Ladder

```
columnas = sorted(triangulo.columns)
factores = {}
for k, k_sig in zip(columnas[:-1], columnas[1:]):
    disponibles = triangulo[[k, k_sig]].dropna().index # años con dato en ambos
    factores[k] = triangulo.loc[disponibles, k_sig].sum() / triangulo.loc[disponibles, k].sum()
```

Con el triángulo del notebook 07, los factores son 1,5963 · 1,1058 · 1,0338 · 1,0135, y la provisión total es de 1.927,7 miles de €. Controles: las celdas observadas no cambian al proyectar, y la provisión del año más antiguo (ya desarrollado) es cero.

PROBLEMAS TÍPICOS

- `max(ST - K, 0)` con un array da `ValueError` : usa `np.maximum` .
- `np.arange(1, n)` genera $n - 1$ pagos: el final no se incluye.
- Resultados distintos en cada ejecución: fija la semilla con `np.random.default_rng(42)` .
- Olvidar el $-\sigma^2/2$ o el descuento e^{-rT} : el precio sale sesgado sin ningún error. Solo lo detecta el caso de control.

AUTOEVALUACIÓN U12

1. ¿Qué dos casos de control usarías para validar una función de rentas?
2. Si duplicas el número de simulaciones, ¿cuánto se reduce el error estándar de Monte Carlo?
3. ¿Por qué el factor de desarrollo usa solo los años con dato en las dos columnas?

Notebooks: [04_teoría_rentas](#) · [05_teoría_opciones_montecarlo](#) · [06_teoría_copulas](#) · [07_teoría_provisiones_chain_ladder](#) · [08_ejercicios](#) · [09_soluciones](#)

PF El proyecto final (opcional)

Un modelo de anulación de pólizas de auto, de principio a fin y sin solución dada. Es el mismo problema que el notebook 02 del Día 10, pero las decisiones son tuyas. Todo el material está en `proyecto_final/` : el enunciado (`README.md`), los criterios de evaluación, un guion de uso de Claude Code por fases y un notebook esqueleto (`notebooks/00_proyecto_anulacion_polizas.ipynb`).

Qué se espera

1. **EDA documentado:** incluye los nulos de `potencia_cv` y el outlier de `prima_anual` , que están ahí a propósito. Explica qué has hecho con ellos y por qué.
2. **Al menos un modelo con Pipeline y ColumnTransformer**, evaluado con métricas adecuadas a un problema desbalanceado.
3. **Matriz de confusión interpretada en términos de negocio:** ¿cuánto cuesta un falso positivo frente a un falso negativo?
4. **Limitaciones y riesgos:** sobreajuste, leakage, variables mal definidas y sesgos, aplicados a *tu* modelo.
5. **Cómo has usado la IA:** qué le has pedido, en qué fase y cómo lo has verificado.

No se pide la mejor métrica posible, sino un trabajo completo, bien razonado y bien comunicado. Consejo: empieza con un modelo simple (regresión logística) que funcione de principio a fin, y mejora después.

PARTE II

Referencia

Para tener a mano durante el curso y después: la política de uso de IA, una guía de errores, chuletas, un glosario, las respuestas de la autoevaluación y cómo seguir aprendiendo.

A Cómo usar la IA en el curso (y en tu trabajo)

Resumen de `docs/uso_ia_en_el_curso.md`. Léelo entero al menos una vez.

La IA generativa se usa en este curso como **herramienta de trabajo sujeta a criterio profesional**, nunca como un atajo para no entender el código. Es el mismo estándar de diligencia que ya aplicas a una hoja de cálculo o a un modelo de un proveedor externo: no firmarías una valoración porque «el Excel lo ha calculado así».

SÍ PUEDES

- Pedir explicaciones de conceptos y de mensajes de error.
- Pedir refactorizaciones, tests y documentación.
- Desatascarte después de intentarlo un rato por tu cuenta.
- Acelerar partes mecánicas que ya dominas.

NO DEBES

- Entregar código que no has leído, entendido y sabrías explicar.
- Resolver con IA el ejercicio de revisión crítica del Día 5.
- Poner en un prompt datos reales de pólizas, asegurados o clientes, ni credenciales.

Cada bloque de IA tiene tres partes

PARTE	QUÉ ES
Prompt sugerido	Para copiar y adaptar.
Qué esperar	Qué debería devolver, para detectar cuándo se desvía.
Qué verificar	El caso de control, la documentación o la comprobación empírica. Es la parte que se evalúa.

Límites que debes tener presentes

- **Alucinaciones:** funciones, argumentos o cifras inventadas con total seguridad.
- **En programación:** código que se ejecuta sin error y calcula mal.
- **En análisis de datos:** la columna equivocada, nulos que desaparecen sin avisar, tipos cambiados. Cuadra totales y tamaños.
- **En modelización actuarial:** un LLM no tiene criterio actuarial. Úsalo para escribir el código de un método que tú ya has decidido, no para decidir el método ni para validar el resultado.
- **Confidencialidad:** trata cualquier asistente en la nube como un tercero. Antes de usarlo con código o datos de tu empresa, consulta su política.

B Guía de errores: síntoma → causa → solución

LO QUE VES	CAUSA PROBABLE	QUÉ HACER
<code>ModuleNotFoundError: No module named 'pandas'</code>	El kernel o la terminal no usan el entorno <code>.venv</code>	Activa el entorno o cambia el kernel del notebook
<code>.venv</code> aparece en «Seleccionar kernel» y desaparece	La extensión de VS Code rehace la lista de entornos una y otra vez	<code>.venv\Scripts\python -m ipykernel install -user --name curso --display-name "Curso Python"</code> y elegir Jupyter Kernel... → «Curso Python»
<code>FileNotFoundError</code> al leer un CSV	La ruta es relativa a otra carpeta	Comprueba con <code>import os; os.getcwd()</code> ; desde un día, <code>"../data/raw/..."</code>
<code>SyntaxError: invalid syntax</code>	Falta <code>:</code> , un paréntesis o unas comillas; <code>=</code> en vez de <code>==</code>	Mira la línea señalada y la anterior
<code>IndentationError</code>	Sangría incoherente	4 espacios por nivel; no mezcles con tabuladores
<code>NameError: name 'x' is not defined</code>	Errata o celda sin ejecutar	Revisa el nombre; Restart & Run All
<code>TypeError: can only concatenate str</code>	Sumas texto y número	f-string: <code>f"Prima: {prima}"</code>
<code>TypeError: 'int' object is not callable</code>	Has llamado <code>sum</code> , <code>list</code> , <code>max</code> ... a una variable	Renómbrala y reinicia el kernel
<code>TypeError: missing 1 required positional argument</code>	Faltan argumentos, o falta <code>self</code> en un método	Revisa la firma de la función
<code>KeyError: 'col'</code>	Esa clave o columna no existe	<code>df.columns</code> , <code>dict.keys()</code> ; ojo con las mayúsculas y los espacios
<code>IndexError: list index out of range</code>	Índice $\geq$ longitud	Los índices van de 0 a <code>len - 1</code>
<code>ValueError: could not convert string to float: '12,5'</code>	Coma decimal	<code>float(x.replace(",", "."))</code> o <code>read_csv(decimal=",")</code>
<code>ValueError: The truth value of a Series is ambiguous</code>	<code>and / or</code> con Series	<code>&</code> , <code> </code> , <code>~</code> y paréntesis
<code>SettingWithCopyWarning</code>	Modificas un subconjunto filtrado	<code>.copy()</code> al filtrar, o <code>df.loc[cond, col] = ...</code>
Resultado con <code>NaN</code> inesperados	Índices que no casan, o un merge sin coincidencias	Revisa índices y claves; <code>indicator=True</code>
Más filas de las esperadas tras un merge	Relación uno a muchos	Agrega antes; <code>validate="one_to_one"</code>

<code>AttributeError: 'X' object has no attribute 'y'</code>	Atributo mal escrito, o falta <code>super().__init__()</code>	Revisa el nombre y el constructor
<code>AttributeError: ... has no attribute 'pdf' (Poisson)</code>	Las distribuciones discretas usan <code>pmf</code>	<code>stats.poisson(mu).pmf(k)</code>
<code>ValueError: Found unknown categories</code>	Categoría nueva en test	<code>OneHotEncoder(handle_unknown="ignore")</code>
El notebook no responde ([*])	Bucle infinito o cálculo muy largo	Interrumpe el kernel (■)
El logging no muestra nada	Nivel por defecto WARNING, o <code>basicConfig</code> ya configurado	<code>basicConfig(level=logging.INFO, force=True)</code>
El cambio en un <code>.py</code> no se ve en el notebook	El módulo está en caché	Reinicia el kernel, o usa <code>%load_ext autoreload</code> y <code>%autoreload 2</code>
<code>ImportError: DLL load failed ... «directiva de Control de aplicaciones»</code>	Smart App Control de Windows 11 o una política de empresa bloquea la librería	Tabla «Si algo falla» de la guía de instalación
Acentos raros (Ñ@)	Codificación	<code>encoding="utf-8"</code> ; para Excel, <code>"utf-8-sig"</code>

Python básico

```
# Tipos
x = 3; y = 2.5; s = "AUTO"; ok = True; nada = None
lista = [1, 2, 3]; dic = {"a": 1}; tupla = (1, 2); conj = {1, 2}

# Conversión
int("3"); float("2.5"); str(3); list(tupla); round(2.567, 2)

# Strings
s.lower(); s.strip(); s.split(";"); s.replace("A", "B"); f"{y:.2f}"; len(s)

# Listas
lista.append(4); lista.extend([5, 6]); lista.pop(); sorted(lista); sum(lista); lista[::-1]

# Diccionarios
dic.get("b", 0); dic.keys(); dic.values(); dic.items(); "a" in dic; dic.update({"b": 2})

# Control
if c: ... elif c2: ... else: ...
for i, v in enumerate(lista): ...
for a, b in zip(l1, l2): ...
[v * 2 for v in lista if v > 1]

# Funciones
def f(a, b=0, *args, clave=None, **kwargs) -> float: ...
lambda x: x * 2

# Errores
try: ... except ValueError as e: ... else: ... finally: ...
raise ValueError("mensaje")
```

pandas

```
df = pd.read_csv(ruta, parse_dates=["fecha"]); df.to_csv(ruta, index=False)
df.head(); df.info(); df.describe(); df.shape; df.columns; df.dtypes
df["col"]; df[["a", "b"]]; df.loc[filas, cols]; df.iloc[0:5, 0:2]
df[(df.a > 1) & (df.b == "X")]; df.query("a > 1"); df.col.isin([...]); df.col.between(1, 5)
df.isna().sum(); df.dropna(subset=["a"]); df.fillna({"a": 0}); df.drop_duplicates(subset=["id"])
df["nueva"] = df.a * df.b; df.drop(columns=["a"]); df.rename(columns={"a": "b"})
df.sort_values("a", ascending=False); df.col.value_counts(); df.col.unique()
df.groupby("g").agg(total=("a", "sum"), media=("a", "mean"))
df.groupby("g").a.transform("sum")
df.merge(otra, on="id", how="left", validate="one_to_one", indicator=True)
pd.pivot_table(df, index="f", columns="c", values="v", aggfunc="sum", fill_value=0)
pd.concat([df1, df2], ignore_index=True); pd.cut(df.edad, bins=[...]); df.fecha.dt.year
```

Excel → pandas

EXCEL	PANDAS
Autofiltro	<code>df[df.col == valor]</code>
Ordenar	<code>df.sort_values("col")</code>
SUMAR.SI(CONJUNTO)	<code>df.loc[cond1 & cond2, "col"].sum()</code>
CONTAR.SI	<code>cond.sum()</code>
PROMEDIO.SI	<code>df.loc[cond, "col"].mean()</code>
BUSCARV / BUSCARX	<code>df.merge(tabla, on="clave", how="left")</code> o <code>df.clave.map(serie)</code>
Tabla dinámica	<code>pd.pivot_table(..., aggfunc="sum")</code>
Subtotales	<code>df.groupby("g").agg(...)</code>
SI / SI anidados	<code>np.where(cond, a, b)</code> / <code>np.select([c1, c2], [a, b], default)</code>
Quitar duplicados	<code>df.drop_duplicates()</code>
Rellenar hacia abajo	<code>df.col.ffill()</code>
Tramos (BUSCARV aproximado)	<code>pd.cut(df.col, bins=[...], labels=[...])</code>

Más en `docs/cheatsheet_excel_pandas.md`.

Git

TAREA	COMANDO
Estado, cambios, historial	<code>git status</code> · <code>git diff</code> · <code>git log --oneline --graph</code>
Guardar	<code>git add archivo</code> → <code>git commit -m "Qué y por qué"</code>
Ramas	<code>git switch -c nueva</code> · <code>git switch main</code> · <code>git merge nueva</code>
Deshacer	<code>git restore archivo</code> (descarta cambios sin guardar) · <code>git revert SHA</code>
Remoto	<code>git clone url</code> · <code>git pull</code> · <code>git push</code>

Más en `docs/cheatsheet_git.md`.

D Glosario de Python

TÉRMINO	SIGNIFICADO
Argumento / parámetro	Valor que se pasa a una función / nombre que lo recibe en la definición.
Array	Estructura de NumPy con elementos del mismo tipo, para calcular rápido.
Atributo	Dato que pertenece a un objeto (<code>poliza.prima</code>).
Broadcasting	Operar arrays de formas distintas sin bucles.
Clase / instancia	Molde / objeto concreto creado con ese molde.
Commit	Una versión guardada del proyecto en Git.
Comprehension	Lista o diccionario construido en una línea con un <code>for</code> .
DataFrame / Series	Tabla de pandas / columna de pandas.
Decorador	<code>@algo</code> encima de una función, que la envuelve.
Docstring	Texto entre triples comillas que documenta una función o clase.
Dunder	Método especial con doble guion bajo: <code>__init__</code> , <code>__repr__</code> .
Entorno virtual	Carpeta con un Python y unas librerías propias del proyecto.
Excepción	Error que interrumpe la ejecución y puede capturarse.
Fixture	Datos o recursos preparados para los tests de pytest.

TÉRMINO	SIGNIFICADO
Índice	Posición (listas) o etiqueta de las filas (pandas).
Immutable	Que no se puede modificar tras crearlo (str, tuple, int).
Iterable	Lo que se puede recorrer con un <code>for</code> .
Kernel	El proceso que ejecuta las celdas del notebook y guarda su estado.
Leakage	Información del test o del futuro que se cuela en el entrenamiento.
Método	Función que pertenece a un objeto (<code>lista.append</code>).
Módulo / paquete	Archivo <code>.py</code> / carpeta de módulos con <code>__init__.py</code> .
ORM	Traduce clases de Python a tablas de una base de datos.
PEP 8	La guía de estilo oficial de Python.
Pipeline	Cadena de preprocesado + modelo que se entrena como una pieza.
Property	Método que se usa como si fuera un atributo.
Scope	Zona del código donde un nombre es visible (LEGB).
Traceback	El informe de un error, que se lee de abajo arriba.
Vectorizar	Operar sobre arrays enteros en vez de hacer bucles.

E Respuestas de la autoevaluación

U1

1. Sí: el kernel sigue recordando `tasa` aunque borres la celda. Con Restart & Run All, las celdas que la usan darán `NameError`.
2. Para aislar las librerías del curso de las del resto del sistema y para que todos tengamos exactamente las mismas versiones, es decir, resultados reproducibles.
3. Que `.media()` no existe (el método es `.mean()`): es una alucinación. Lo compruebas ejecutándolo (`AttributeError`) o con `help` o la documentación.

U2

1. `[1, 2, 3]`, porque `b` y `a` son la misma lista. Con `b = a.copy()`, `a` sigue siendo `[1, 2]`.
2. `'100'` (posiciones 3, 4 y 5).
3. Porque las claves deben ser inmutables (hashables): la tupla lo es y la lista no.
4. `f"{1234.5:,.2f}"`.

U3

1. 2, 5, 8.
2. `[p * 1.05 for p in primas if p > 300]`.
3. No has ejecutado la celda que define `cartera` desde el último reinicio del kernel, o hay una diferencia en el nombre.

U4

1. `None`: calcula `x * 2` pero no lo devuelve.
2. `def valor_actual(capital: float, i: float, n: int) -> float: con su docstring y return capital / (1 + i) ** n.`
3. Un parámetro sin valor por defecto no puede ir detrás de uno que lo tiene. Correcto: `def f(prima, recargo=0.1).`
4. `sorted(polizas, key=lambda p: p["edad"])`.

U5

1. Todas las pólizas pasan a usar 0,10, porque `prima_total` lee `self.TASA_IPS` cada vez y ese atributo es de la clase (salvo que alguna instancia tenga su propio `TASA_IPS`).
2. Porque es una `@property`: se define como método pero se usa como atributo.
3. Que todas las instancias compartan la misma lista por defecto. Cada objeto recibe su propia lista vacía.

U6

1. Cuando el bloque `try` termina sin lanzar ninguna excepción.
2. Porque los floats tienen errores de redondeo y `==` puede fallar aunque el resultado sea correcto a efectos prácticos.
3. `git add` prepara los cambios (staging); `git commit` los guarda como una versión en el historial.
4. Porque contiene credenciales: si se sube a Git, quedan expuestas y permanecen en el historial.

U7

1. `loc[0:3]` selecciona por etiqueta e **incluye** el 3 (4 filas con el índice por defecto); `iloc[0:3]` selecciona por posición y lo excluye (3 filas).
2. `cartera[(cartera.producto == "HOGAR") & (cartera.prima_anual > 300) & (cartera.fecha_baja.isna())]` .
3. `cartera.groupby("canal_venta").anulada.mean()` .
4. La relación era de uno a muchos (por ejemplo, varios siniestros por póliza): cada póliza se ha repetido. Agrega antes o usa `validate` .

U8

1. El objeto `Axes` : un gráfico concreto dentro de la figura, sobre el que dibujas y pones títulos y ejes.
2. Un `boxplot` (o `violinplot`) con `x="producto"` e `y="importe_pagado"` .
3. Si muestra la media (por defecto) o la suma, qué significan las barras de error y si el eje Y empieza en cero.

U9

1. Es la clave foránea: enlaza cada siniestro con su póliza en la tabla de pólizas.
2. `session.commit()` .
3. El token: si existe, si se ha cargado (`load_dotenv`), si ha caducado y si la cabecera `Authorization` está bien escrita.

U10

1. `.ppf(0.95)` (o `.isf(0.05)`).
2. La Poisson es discreta: tiene `pmf` , no `pdf` .
3. Un chi-cuadrado de independencia (`chi2_contingency`) sobre `pd.crosstab(df.forma_pago, df.anulada)` .

U11

1. Para que train y test tengan la misma proporción de anulaciones, algo importante cuando la clase es minoritaria.
2. `OneHotEncoder` lanza un error al predecir, porque no conoce esa categoría.
3. El recall. Bajas el umbral de decisión (por ejemplo, de 0,5 a 0,3): detectas más anulaciones a cambio de más falsos positivos.

U12

1. Por ejemplo: la fórmula cerrada $(1 - \sqrt[n]{n})/i$, y que con n muy grande tienda a la perpetua $1/i$. También sirve que con $i \rightarrow 0$ el valor tienda a n .
2. El error estándar es $s/\sqrt{N}$: al duplicar N se divide por $\sqrt{2}$, es decir, baja un 29 % aproximadamente. Para reducirlo a la mitad hay que cuadruplicar N .
3. Porque el factor compara el mismo año de origen en dos periodos consecutivos. Si un año no tiene todavía el periodo siguiente, no aporta información sobre ese paso.

F Cómo seguir aprendiendo

- **Haz el proyecto final.** Es el mejor siguiente paso: aplicar todo, de principio a fin, sin red de seguridad.
- **Lee código real** antes que tutoriales nuevos: los notebooks del curso y la documentación de pandas, NumPy, SciPy y scikit-learn, que incluye muchos ejemplos.
- **Empieza por la documentación oficial** cuando tengas una duda concreta. Es mejor fuente que un blog, y puedes pedirle a Claude Code que te la resuma o la contraste.
- **PEP 8 y pytest no son solo del Día 6:** son el mínimo profesional a partir de ahora.
- **Practica con tus propios problemas de trabajo**, siempre con datos sintéticos o anonimizados. La distancia entre «lo entiendo en el ejercicio» y «lo aplico a mi caso» solo se cierra haciéndolo.
- **Vuelve a los `README.md`** de cada día del repositorio: están pensados como referencia rápida para después del curso.

Para profundizar

- Wes McKinney, *Python for Data Analysis* (3.^a ed.), del autor de pandas, disponible en abierto en su web.
- La guía de usuario de scikit-learn, en especial los apartados de *Pipelines* y *Model evaluation*.
- Wüthrich y Merz, *Statistical Foundations of Actuarial Learning and its Applications* (Springer, 2023, acceso abierto), para GLM y ML aplicados a seguros.